Database Report

Junzhang Li (Group 8)

March 10, 2025

1 Account of database design and SQL

The movie dataset referenced in this database contains a large number and variety of song metadata, which are mainly arranged by movie name and user name, and contain large number of duplicate data. This poses a great challenge for the performance of data screening as well as data storage. To address this issue, we designed a database schema for the original movie data, aimed to minimise redundancy and dependency by organise movie data into well-structured tables. Figure 1.1 expresses our database schema in the form of Entity-Relationship Diagram(ERD):

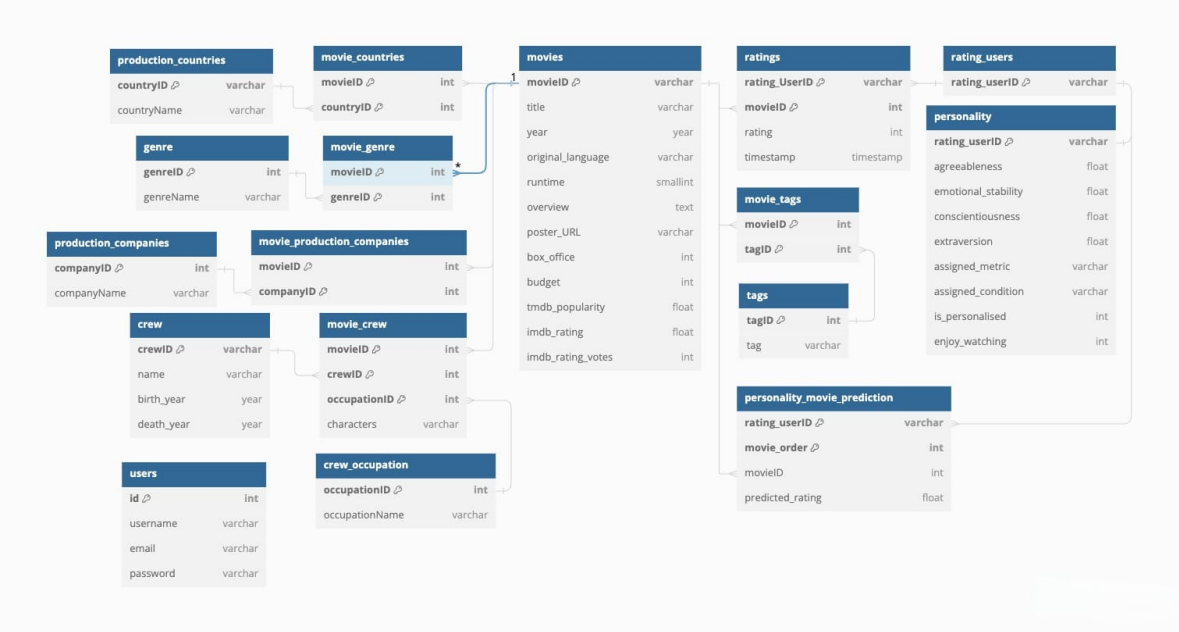

Figure 1.1: Entity Relationship Diagram of our database

Here, different states of normalisation for different data types are explained:

1.1 First Normal Form (1NF)

First normal form is applied for all tables contain atomic values, with no repeating groups.

- `movies`, `production_countries`, `genre`, `production_companies`, `crew`, `crew_occupation`, `rating_users`, and `tags` tables all have atomic attributes.
- Composite primary keys are utilised where necessary, such as in junction tables like `movie_crew`, `movie_genre`, `movie_countries`, `movie_production_companies`, `ratings`, `movie_tags`, and `personality_movie_prediction`.

1.2 Second Normal Form (2NF)

In second normal form, a table is in 1NF and does not have partial dependencies on the primary key. In this schema:

- No non-prime attributes are dependent on only a portion of a composite primary key.
- For instance, attributes like `title`, `release_year`, `original_language`, `runtime`, `overview`, `poster_URL`, `box_office`, `budget`, `tmdb_popularity`, `imdb_rating`, `imdb_rating_votes` in the `movies` table depend solely on the primary key `movieID`, not on any subset of it.

1.3 Third Normal Form (3NF)

Our database schema also implies third normal form, a table is in 2NF and does not have transitive dependencies. In this schema:

- Attributes are not transitively dependent on the primary key.
- For example, `genreName` in the `genre` table depends directly on the primary key `genreID`, not on any non-prime attributes.

1.4 Discussion

The provided database schema is well-normalized up to 3NF. It effectively eliminates redundancy and ensures data integrity by structuring data into distinct tables and establishing appropriate relationships between them. Key points contributing to normalization include the use of composite primary keys in junction tables, proper identification of primary keys, and avoidance of partial and transitive dependencies.

1.5 Conclusion

Normalisation is essential for efficient database management, pivotal for maintaining data consistency, minimising redundancy, and streamlining maintenance procedures. This database schema showcases a commendable adherence to normalisation principles, upholding a robust structure compliant with Third Normal Form (3NF). This underscores its capability to efficiently manage and manipulate data. Consequently, the schema lays a solid foundation for conducting further analyses on diverse types of movie data, including evaluation in correlation across them.

2 Explanation and critique of requirements

2.1 Requirement 1

We implemented a powerful and high aesthetic dashboard page and movie details page.

The dashboard can display Poster, Title, Year, Original Language, Runtime, Box Office, Budget, TMDB Popularity, IMDB Rating, IMDB Rating Votes, Country, and Genre according to user needs. Where the link of the poster stored in the database is displayed as the pointing image on the dashboard front end.

In addition, the system only requests the data information required by the user, rather than filtering the data after requesting all the information, which reduces the operating pressure of the database server and improves the request performance.

Note that the database server can capture all the information of all the movies very quickly, but rendering all this data takes a lot of time and performance, potentially causing the User Interface lags. By introducing pagination, we solved this problem, where each page in our dashboard shows 25 movie entries, and users can switch pages to see more movie entries.

2.2 Requirement 2

In the searching page, we offer two main search concepts, precise search and fuzzy search, which provides users with more diversified searching patterns.

The former requires users to select specific search objects, covering the title, director, lead actor, film genre, country, release date and distribution company. In the latter case, the search object is defined as any of the aforementioned objects, and the LIKE operator is applied to find movie entries with a "search term" in any location for display. Here is the searching box in our application:

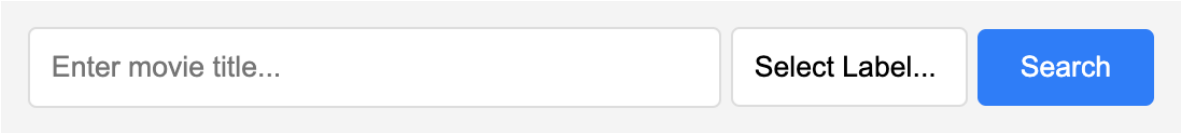

Figure 2.2: Design of input boxes for requirement 2

2.3 Requirement 3

2.3.1 Implementation - 1

In this requirement, we provide independent analysis reports on the raw data set, TMDB data set, and IMDB data set because the data formats are not mutually exclusive. Each movie genre in each report provides a visual representation of the relevant data using bar charts, resulting in greater readability.

For the original dataset, we provide two approaches of browsing the most popular genres, based on the total score of the genre and the proportion of high ratings (ratings greater than 3.5) in descending order, and use a bar chart to show the proportion of high, medium, and low ratings for each genre.

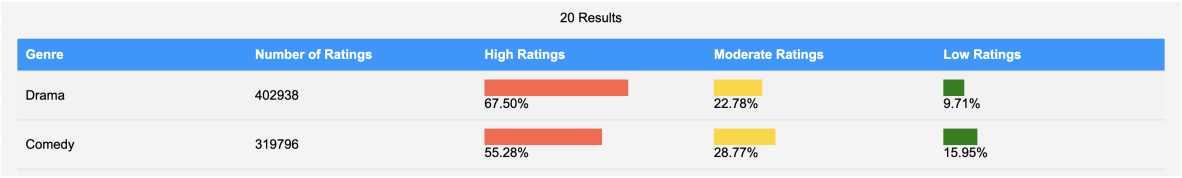

Figure 2.3: Design of Most Rated genre page

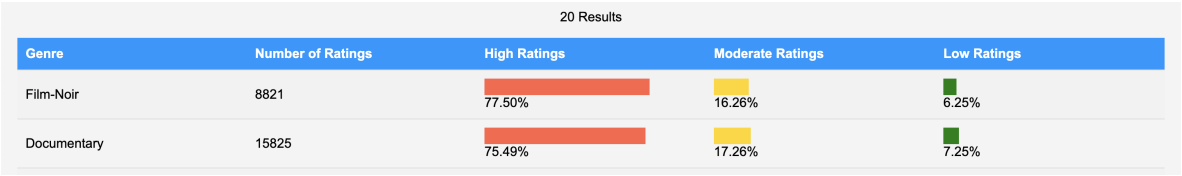

Figure 2.4: Design of Highest Rated genre page

2.3.2 Query - 1

The SQL query for these two pages computes statistics on movie ratings categorised by genre. It first calculates the total number of ratings and the percentage of ratings falling into three categories: low (below 2.5), mid (between 2.5 and 3.5), and high (above 3.5) for each genre. These calculations are performed in a common table expression (CTE) named RatingStats. Then, it joins the result of the CTE with the genre table to retrieve the genre names. Finally, it presents the genre names along with the total ratings and the percentage of ratings falling into each category, ordered by the total number of ratings and the percentage of high ratings in descending order, respectively.

2.3.3 Implementation - 2

For most polarised movie genres, we use query instructions to calculate the mean squared error of all scores for each type of movie, which is used to quantify how far the average rating deviates from that mid rating (3). On this page, the genres are listed in descending order of the mean squared of their ratings, and the percentages from 1 to 5 are shown, giving users a more specific look at the form of polarisation.

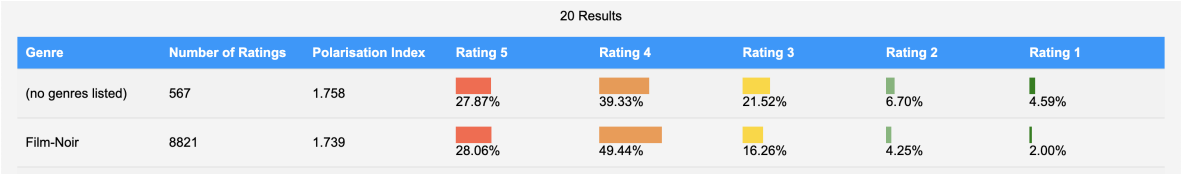

Figure 2.5: Design of Most Polarised genre page

2.3.4 Query - 2

The SQL query for this requirement have a similar structure to the query for most rated/highest rated genre, it adds a new column to evaluate the mean square error for each genre and ordered by this metric in descending order.

2.3.5 Implementation - 3

In addition, we also provide TMDB and IMDB data display. Due to the limited data types, the former only provides the ranking with the highest rating and the most ratings, while the latter only provides the ranking with the highest popularity, with no polarisation data for ratings in both table.

All available data are clearly displayed using bar charts.

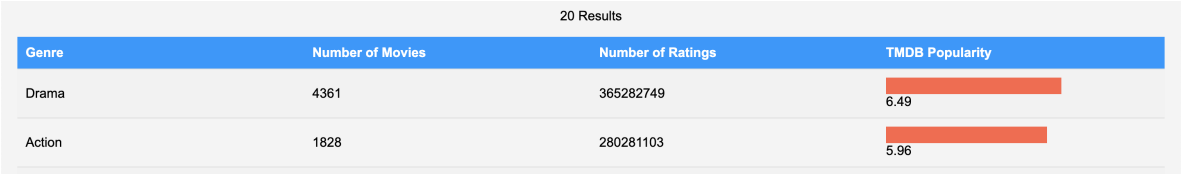

Figure 2.6: Design of Most Rated genre page for IMDB

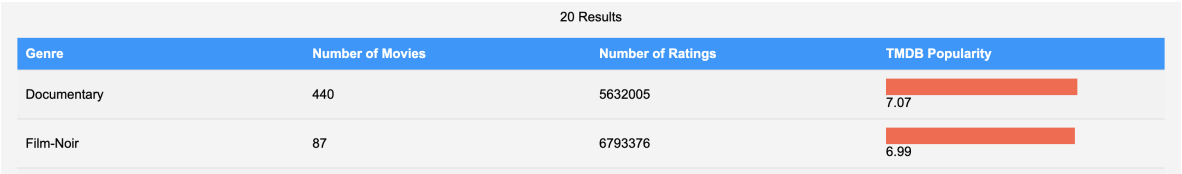

Figure 2.7: Design of Highest Rated genre page for IMDB

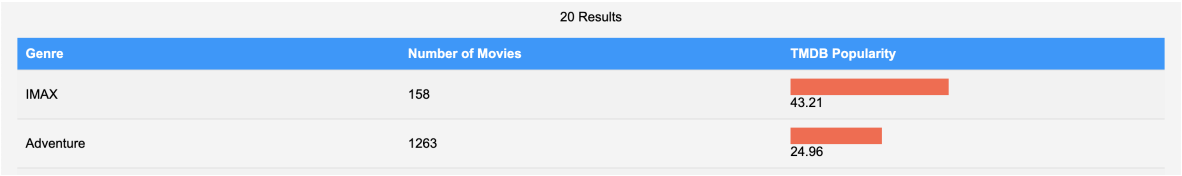

Figure 2.8: Design of Highest Popularity genre page for IMDB

2.3.6 Query - 3

IMDB and TMDB provide a rating data for each movie directly, thus for their queries, we simply count the average rating and number of rating for each movie, categorised into genres, and order in descending order.

2.3.7 Evaluation

High degree of completion.

On the front end, we use a bar chart to illustrate the distribution of ratings for each movie genre, allowing users to dig deeper into the popularity and polarisation information.

In terms of query structure, all of our calculations and ordering are done in query, including the calculation of the mean square error (the degree of polarisation), and the back-end is only responsible for directly printing the results of the query instructions.

In term of query efficiency, our queries also perform well, with the rating information for each movie being summarised by CTE first, and categorised into different genres for final output.

2.4 Requirement 4

2.4.1 Implementation - 1

In this part, we aim to find the relation between a group of people with same rating trend and their ratings toward a given movie. Here is the fixed format for user input:

Figure 2.9: Design of input form for requirement 4.1

2.4.2 Query - 1

The query for this part find group of users who gave a rating to chosen movie, and find specific group of users with average rating for all movies in the given range. Eventually display the rating information from these specific group of users.

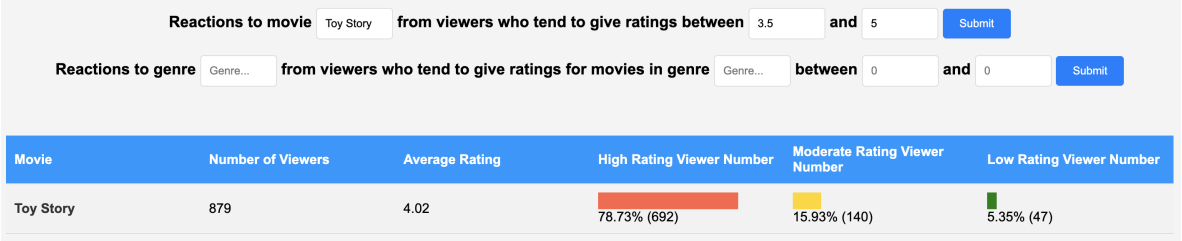

Figure 2.10: Example of a input for requirement 4.1 and its output

Figure 2.10 illustrates that there are 1172 viewers, who has average rating for all movies between 2.5 and 5, rated movie "Toy Story". Of these viewers, most of them gave this film high rating (78.73%), indicates that:

Viewers who tend to give high ratings give film "Toy Story" a high rating too!

2.4.3 Implementation - 2

In the second part of requirement 4, we improved the algorithm of query to find the relationship between two genres, Here is the fixed format for user input:

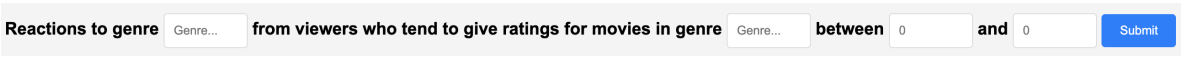

Figure 2.11: Design of input form for requirement 4.2

2.4.4 Query - 2

This query looking for a group of users gave the average rating for one genre in a user-defined range, and evaluate their rating information toward any films in another genre.

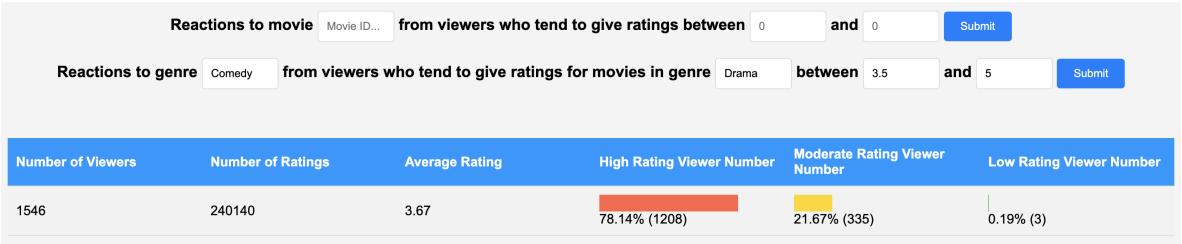

Figure 2.12: Example of a input for requirement 4.2 and its output

Figure 2.12 shows that in the group of viewers who give a average rating between 3.5 and 5 for Dramatic movies, most of them tend to give a high ratings toward comedic movies (76.8%). This indicating that:

Viewers who tend to rate Drama films highly also rate Comedy genre highly!

2.4.5 Evaluation

High degree of completion. Both queries in this requirement have a high performance.

For the first part, instead of finding people with same rating trend, and then extract those have rated given movie, we find group of people who rated given movie first, and then find the group of people with same rating trend.

This is because there are much more rating entries than movie entries, by extracting viewers who gave a rating to chosen movie, we can narrow the number of view to be further filter quickly, therefore provide a good performance for the query.

For the second part, we found a subset of users who have rated genre 1 movies within the set of users who have rated genre 2 movies in specific range, but not searching for user having rated genre 1 and genre 2 and get the intersect users. This brings a better performance especially when there are large number of rating for movies in two selected genres.

It is worth notice that both queries takes a few second to run, where the time cost of finding the target group of users is extremely low. The most time-consuming action is counting the number of viewers who give a high/moderate/low ratings. Therefore, to shorten the query time, moderate ratings are evaluate by the rest of viewers neither give high rating nor low ratings, instead of looking for the number of people give moderate rating from the table.

2.5 Requirement 5

2.5.1 Implementation - 1

For this requirement, A range of viewers is selected by user, and their ratings towards a given movie are averaged, applied as the prediction for the overall average rating. Figure 2.13 shows the fixed format for user input:

Figure 2.13: Design of input form for requirement 5.1

For this user input, it's necessary to explain the "active users", where a user giving more ratings is recognised to be more active. Here, we selected a range of most active users because it is believed that the active users is likely to give a more unbiased rating compare to the inactive users. Therefore, their scores should be representative and close to the overall movie rating.

2.5.2 Query - 1

In this query, the set of viewers who have rated chosen movies will be identified first, then the total number of ratings given by each viewer in this set is evaluated, and sorted the viewers in descending order according to this number. In our words, the table generated so far lists all users who rated the chosen movie, ordered from most active user to most inactive user. Now, the percentage value defined in the input is used to select the previous (percentage)% row of the user table. And eventually their average rating toward this movie is set as the prediction for overall rating. Here is an example input and corresponding prediction:

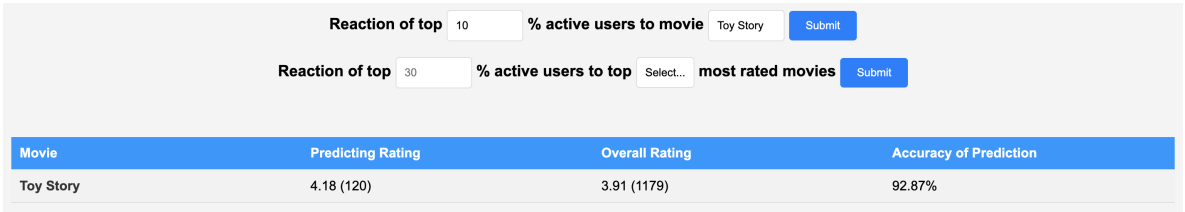

Figure 2.14: Example of a input for requirement 5.1 and its output

2.5.3 Implementation - 2

The second task in this requirement is achieved using the same algorithm, except that the movie being analysed is the top 1 to 10 most rated movies, as shown in figure 2.15

Figure 2.15: Design of input form for requirement 5.2

2.5.4 Query - 2

Compare to the previous query, a new table is defined at the beginning, to illustrate the top 1-10 most rated movies. Then these movies are iterated to evaluate the prediction rating, overall rating and accuracy. Here is a screenshot for example input:

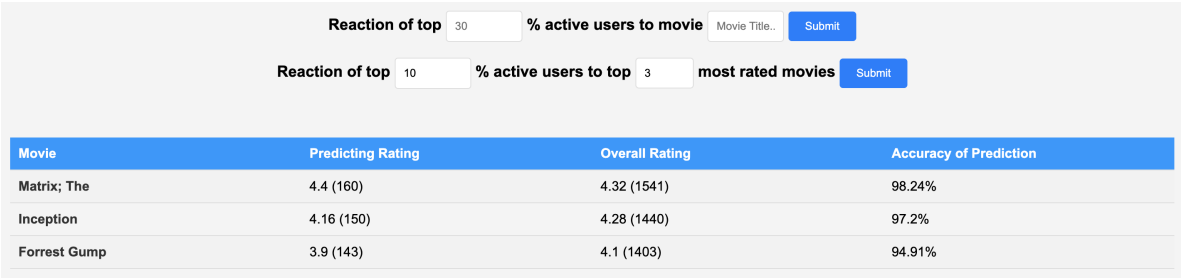

Figure 2.16: Example of a input for requirement 5.2 and its output

2.5.5 Evaluation

High degree of completion.

Notice that **evaluation for all columns for every movie is implemented in a single query**, php is only responsible for generating single query with user input and output the result directly.

Beside, queries in this requirement have a low complexity as well. Activeness is only evaluated for the viewer who gave a rating to the selected movies, but not for all existed viewers, which greatly reduce the complexity of query.

2.6 Requirement 6

2.6.1 Implementation - 1

For this requirement, we applied Pearson correlation coefficient to quantify the relation between the personality traits and preferences(average rating for all movies).

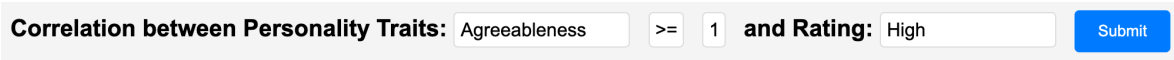

Figure 2.17: Design of input form for requirement 6.1

2.6.2 Query - 1

In this query, we set two criteria, trait and average score. And use these metrics to filter out eligible users. Then, by calculating the average traits and average scores, all the data are incorporated into the Pearson correlation algorithm to determine whether there is a correlation between the selected personality traits and the movie rating tendency.

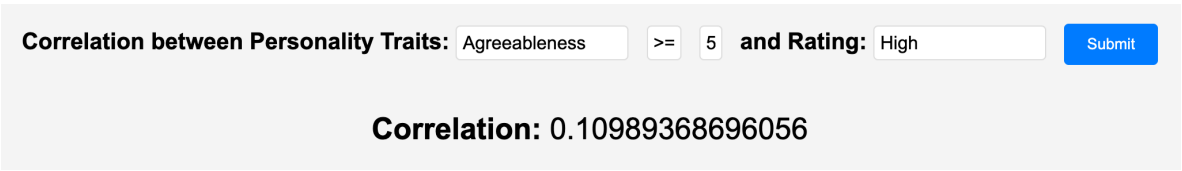

Figure 2.18: Example of a input for requirement 6.1 and its output

2.6.3 Implementation - 2

This part of the query using a similar algorithm to the implementation 1, with the user range being modifies only.

Figure 2.19: Design of input form for requirement 6.2

2.6.4 Query - 2

Instead of screening the set of users whose overall average score meets the requirements, the query here filters users with specific preferences for specific movie types, and further filters out users with similar personality characteristics to calculate their preferences.

Figure 2.20: Example of a input for requirement 6.2 and its output

Through this method, we can determine whether there is a correlation between personality traits and preferences for specific genres of movies.

3 Review of design effectiveness and applications opera¿on

3.1 Value to Users

The Movie Database web program presents an unparalleled solution for movie enthusiasts for users. Here's a detailed explanation of the value the platform brings to the film community:

- **Data accuracy.** The database MovieLens, backed by GroupLens, a research lab at the University of Minnesota, ensures the accuracy of the movie data. In addition, the database covers a vast amount of metadata on nearly 10,000 films from the early twentieth century to today, covering different genres, directors, countries, languages. The great variety ensures that users will always find the movie they want.
- **Excellent user interface design.** Dashboards, movie details pages, and search pages are all designed to highlight posters, complemented by detailed information, in a way that is more user-friendly than similar products and promotes user interest in using the platform. For more professional user rating analysis and movie type analysis, the data is always displayed in a bar chart as the main element, both data analysis experts and ordinary users can find their own interesting content in a simple and efficient analysis interface.
- **Ease of use and security.** All components of the network application are packaged with Docker, which greatly facilitates the distribution and maintenance of the application, and does not make any requirements on the user's system. Users do not need to perform tedious code operations, just install Docker, run Docker Compose command, and enjoy the service. And compared with other competitive products, the program is completely offline, does not pose any threat to user privacy.
- **User engagement and satisfaction.** At the bottom of the login screen is the contact information of the development team. We encourage users to actively express their opinions, which helps improve user engagement and the pursuit of better apps.

3.2 Security Mechanism

The network programme uses extensive security to guard against external intrusion. In particular:

- **Input Specifications.** In this web application, we have reduced the text input as much as possible, without affecting the functional implementation and user experience. Specifically, for

functions with complete requirements and able to run without user definition, buttons are used for interaction, these reflected in the displayed column selection in home page and fixed options in Q3. For a feature that requires the user to decide between several choices, we use drop-down menus for interaction, such as the select box to define search target in the search page, and the selection of the number of movies with the most ratings in Q5. For inputs that require user definition on a larger scale, we strictly regulate input types and conditions. For example, for percentages in Q5 and ratings in Q4, only numeric inputs are accepted, and are limited in range and precision. For functions that do require text input, the section below explain the safety scheme.

- **Preventing SQL Injection and Cross-Site Scripting (XSS) Attacks.** In this web application, all the functions that need to use user text input to implement query instructions are effectively protected against SQL injection attacks and XSS attacks.

  To protect web app from the SQL injection attack, the prepare statement is applied in the generation of each query. Specifically, prepare statement separates SQL queries from user-input data, helping to prevent SQL injection attacks by automatically escaping special characters in input data such as '/' and ';'.

  To protect web app from the XSS attacks, htmlspecialchars are used to sanitise input data, converting special characters into their HTML entities, to prevent them from being interpreted as HTML or JavaScript code.

- **Restrictions on URL Access.** Most of the functionality reads user input in the web page, except for the movie details page, which identifies the movie based on the end of the URL. Here's an example of the URL for a movie details page:

  http://localhost:4000/movie_details.php?id=1

  In our program, the movie ID is read from the URL, which cannot impose input restrictions in this regard. Considering that the ID used in this database is not industry common, and there is no front-end page to provide movie ids to users, users directly modify the URL to query relevant movie information is not in line with design expectations. To address this potential vulnerability, the presence of `HTTP_REFERER` is checked in the server's environment variables before accessing the details page. If this variable is empty or does not exist, the program will stop displaying the movie details. This method prohibits the possibility of accessing the details page by modifying the URL, ensuring that the user can only access the movie details page through the relevant link given by our system.

3.3 Scalability Mechanism

To accommodate more potential users, the Web application's scalability design has been significantly enhanced.

- **Public Gateway - Nginx** In our development setup, we effectively solve the need for a public gateway service and the isolation of other components and services within a virtual private network using Nginx. Nginx serves as the entry point for external connections, listening on port 4000, and efficiently routes these requests to the appropriate back-end services. These back-end services are isolated within the Docker container network, ensuring their accessibility only through the gateway provided by Nginx. By leveraging Nginx as a reverse proxy, we centralise and manage incoming traffic effectively, enhancing security by abstracting the internal network structure and providing a single point of entry for external requests.

- **Dynamic Load-Balancing Mechanism in Back-end Servers.** By using Nginx as the reverse proxy server, we implement a powerful load balancing mechanism using the upstream directive. This directive distributes incoming requests to multiple back-end servers, ensuring efficient resource utilisation and improved response times. With the ability to handle large numbers of concurrent connections, our systems can scale seamlessly to meet growing demands without sacrificing performance. In addition, the inclusion of proxy buffering and error handling mechanisms, such as `proxy_next_upstream`, enhances reliability by handling errors and ensuring uninterrupted service delivery.

- **Dynamic load-balancing Mechanism in Database Servers.** With the configuration of HAProxy, our scalability design for database management has achieved a significant breakthrough. By implementing a dynamic load-balancing mechanism within the mysql_servers backend, we ensure optimal distribution of incoming MySQL traffic across multiple database servers. Utilising the round-robin algorithm, each incoming connection is evenly distributed among the db1, db2, and db3 servers. This not only enhances the system's capacity to handle a larger number of concurrent database connections but also improves overall reliability and fault tolerance. In addition, by implementing the health check function in HAProxy, the availability of each database server is continuously monitored, allowing automatic removal and reintegration of servers according to the health status of the server, effectively avoiding long-term server breakdown due to potential system problems.

With these advancements in scalability design, our web applications deliver superior performance and user experience even under heavy load.

4 Record of personal contribution

4.1 Week 2

4.1.1 Achievements

A large number of data sets were found and compared, data quality and potential usability and problems were evaluated respectively, and the data sets needed to be used were determined.

4.2 Concrete Endeavour

This week I learned about the large number of movie datasets that can be utilised, and for the large number of databases available in MovieLens, ml-latest-small was chosen as the initial dataset due to the difficulty of processing large datasets and the runtime. We also studied a large number of rating datasets from different movie databases, among which we noted that Rotten Tomatoes' datasets used completely different movie ids compared to IMDB and TMDB, which presented a challenge for the integration of movie data, so we only used TMDB and IMDB datasets as a supplement to the rating data.

Considering that we need to conduct research on user personality, we also learned about the Personality-ISF2018 dataset. One notable issue is that both this dataset and ml-latest-small contain user rating data, but the user ids are written in completely different ways (hashes and integers). Considering that the user personality data in the Personality-ISF2018 is just needed, we use the user rating data table in the database.

4.3 Week3 & 4

4.3.1 Achievements

Completed learning all components and language needs to be applied in this project. Includes Docker initialisation, learning and comparison of web servers (Apache and NGINX), learning and comparison of data storage engines (InnoDB, MongoDB).

4.4 Concrete Endeavour

Through the videos in linkedin and Microsoft Learning, I understood the concept and basic setup of Docker. For each component that is learned later, its application methods in Docker are also learned simultaneously.

Web server learning focuses on Apache and NGINX, which are the two most mainstream methods at present. By learning and comparing them separately, I realized that NGINX has an asynchronous, event-driven architecture compared to the more traditional Apache, which allows it to handle high concurrency and scale more efficiently. In addition, it makes it easier to set up some scaling mechanisms, which are necessary components in future development. With the above arguments in mind, I decided to use NGINX as our back-end server.

For the database engine, I prefer to use InnoDB. Admittedly, MongoDB, as a NoSQL database, is advantageous for programs with rapidly evolving data structures and high scalability requirements. But InnoDB, as the default database for MySQL, does not require additional setup, and is easier to get started with, which is why I ultimately chose it.

4.5 Week5

4.5.1 Achievements

Set up the basic format of each requirement in the web application, as well as the initialisation of InnoDB.

4.6 Concrete Endeavour

This week, I built a simple web application. Docker is used to set up the back-end server and the database server respectively, and the data interaction is realised through the correct connection interface. In the previous pair, I created a blank page for each task and achieve the redirect between pages by setting the navigation bar.

4.7 Week6 - Reading Week

4.7.1 Achievements

Cooperated to complete the database cleaning and normalisation.

4.8 Concrete Endeavour

Prior to this, the rest of the team had been working on the data collection, but due to the large dataset of the film, it needed more time to complete. Therefore, I also joined the database sorting work, mainly responsible for the standardised processing of movie data in ml-latest-small. Finally, we completed the cleaning of TMDB and IMDB data of movie data and score data, and imported the movie data into InnoDB and sent it to the front end through the network server for display.

4.9 Week 7-10

4.9.1 Achievements

Complete the home page (Requirement 1) and Requirements 3 to 5.

4.10 Concrete Endeavour

In weeks 7 to 9, I focused on functional development, including the setup of Query and the presentation of the results in PHP. In week 10, by group discussion, I improved the implementation of some features, including the introduction of pagination on the home page, reducing rendering pressure and reducing latency, and implement the specification of all user input.

4.11 Week 11

4.11.1 Achievements

Review web application security mechanisms and introduce scaling mechanisms

4.12 Concrete Endeavour

For web pages and scaling mechanisms, I learned a lot of cutting-edge processing patterns, the former including protection against injection attacks and cross-site scripting attacks, the latter including load-balancing, caching, micro-services and other methods. All of the methods eventually implemented are fully described in Chapter III of this report.